Final Project Proposal

Team Members:

- Chen-Yu Fan
- Po-Chun Liu

Table of Contents

1. Introduction to the LED Piano
2. Structural Design
3. Mechatronic Integration
4. Scoring System
5. Challenges Encountered and Solutions
6. Conclusion

1. Introduction to the LED Piano

User Side:

LED Indicator Display → Press the Piano Key According to the LED Signal → Score Display

Signal Side:

Control Signal from Computer to LED (On/Off Control Signal) → Touch Sensor → Seven-Segment Display for Score

The Arduino is responsible for generating the voltage signals that control the LED indicators. The NI ELVIS system acquires the voltage signals from both the touch sensor switches and the final LED in the sequence. It then outputs control signals to the seven-segment display and the computer speaker.

Since the LED sequence operates independently of the user input, the LED signals are directly generated by the Arduino. Only the voltage signal of the last LED is transmitted to the ELVIS system for score evaluation and judgment.

Furthermore, no communication protocol is required between the two microcontrollers, as they operate independently with minimal signal interaction.

2. Structural Design

1. Piano Keys

Materials:

White keys: Paper clay
Black keys: Resin clay

Tools:

Steel ruler
Plastic folder (used as a base plate)

Method:

The piano keys were shaped by pressing and molding the clay using a steel ruler, forming the structures shown in Figure 1 and Figure 2. For the white keys, an additional small elevated support piece was fabricated. This elevation structure is essential for implementing the key return (rebound) mechanism, which will be explained in detail in the next section.

Figure 1

Figure 2

2. Key Return Mechanism

Materials:

Corrugated board
Elastic rubber band (appropriately sized, similar to a hair tie)

Tools:

Utility knife
Adhesive tape

Method:

As shown in Figure 3, a U-shaped cut was made in the corrugated board using a utility knife. The cut section was then lifted upward to form a flexible structure.

Next, an elastic rubber band was looped around the raised U-shaped section. The U-shaped structure was then pressed back into its original position, as illustrated in Figure 4.

Finally, adhesive tape was applied to secure the corrugated board and prevent the U-shaped section from lifting again, which could cause the rubber band to shift out of position.

The completed return mechanism is shown in Figure 5.

Figure 3

Figure 4

Figure 5

3. LED Strip and Seven-Segment Display

Materials:

WS2812 LED strip
Corrugated board
Seven-segment display

Tools:

Adhesive tape
Utility knife
Saw
Tweezers

Method:

First, a rectangular base plate was cut from the corrugated board using a saw.

Next, holes were carefully created at the positions corresponding to the LEDs on the strip and the seven-segment display using tweezers, as shown in Figure 5.

The WS2812 LED strip was then attached to the board using adhesive tape, as illustrated in Figure 6. The seven-segment display was inserted into the two square openings located at the upper section of the board and secured with tape.

Finally, the black keys were attached to the lower section of the board. Upon completion, the mounting structure for the LED strip and the seven-segment display was established, as shown in Figure 7.

Figure 5

Figure 6

Figure 7

4. Touch Sensor and Start Button

Materials:

Capacitive touch sensor
Push button switch
Adhesive tape
Corrugated board

Tools:

Scissors
Electrical wires
Soldering iron
Utility knife

Method:

Electrical wires were first cut to appropriate lengths using scissors. The terminals of the capacitive touch sensor were soldered to the wires using a soldering iron to ensure stable electrical connections.

The wires were then routed from the bottom of the piano structure to the rear side, where they were connected to a centralized breadboard for signal integration. The wiring was secured with adhesive tape, as shown in Figure 8.

Next, a small square opening was cut into the corrugated board using a utility knife. The push button was installed into the opening and tightly fixed in place. Its wiring was similarly routed to the rear breadboard for circuit integration.

The completed assembly is shown in Figure 9, where the breadboard is positioned between two corrugated boards for structural support and protection.

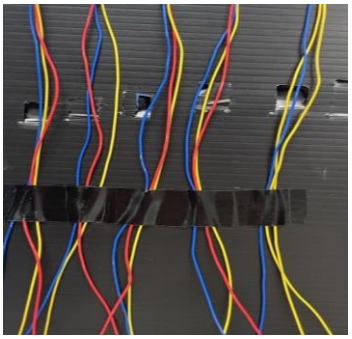


Figure8

Figure 9

3. Mechatronic Integration

Touch Sensor Switch Control Principle

1. Installation:
DuPont wires were used to connect the capacitive touch sensor to the analog input channels of the NI ELVIS system.

2. Control Mechanism:
When a piano key is pressed and makes contact with the touch sensor switch, the sensor is activated and outputs a continuous voltage signal. This voltage signal is acquired through the DAQ Assistant module in LabVIEW.

The acquired voltage value is then used as a logical condition for system control and integration with other components. In other words, once the touch sensor detects a valid touch signal, the corresponding system components can immediately respond based on the predefined control logic.

LED Strip

1. Design Concept:
Seven rows of LED strips represent the seven piano keys. The illumination pattern (on/off states) corresponds to the musical score.

Different colors are used to indicate specific keys and different rhythmic patterns, guiding the player during gameplay. When the illuminated signal reaches the final row, the user is required to press the corresponding piano key.

2. Implementation Method:

(1) WS2812 LED

The system utilizes a WS2812 LED strip. The WS2812 is a programmable RGB LED integrated within a 5050 package, which includes a built-in driver IC. This integrated design provides high controllability and enables seamless continuous lighting effects along the strip.

Each 5050 LED package contains red, green, and blue LEDs. By adjusting the brightness values of these three color channels, a wide range of colors can be generated to achieve various visual effects.

In addition, the WS2812 utilizes a single-wire serial data interface. Developers can control multiple LEDs simultaneously through a single data pin, thereby significantly simplifying the wiring and control complexity of the system.

For the programming implementation, the FastLED library in Arduino was adopted. Control logic was implemented using loops and conditional statements to individually address and manage each LED. Through this approach, the LEDs were programmed to illuminate

according to the musical notes and rhythm of the score, achieving synchronized visual guidance for the player.

(2) Program Control

● CRGB Object

As shown in the figure below, within the FastLED library, each LED is declared as a CRGB object. The brightness and color of each LED are determined by specifying the red, green, and blue intensity values within the parentheses.

Since the system was designed with seven piano keys, each assigned a distinct color, individual CRGB color definitions were first established for each key. This allowed precise control of the visual representation corresponding to each musical note.

```
#define highd CRGB(15, 0, 10)
#define highr CRGB(15, 4, 0)
#define highm CRGB(15, 10, 0)
#define highf CRGB(0, 25, 0)
#define highs CRGB(0, 0, 25)
#define highl CRGB(10, 0, 10)
#define highc CRGB(10, 0, 10)

#define high2 CRGB(10, 5, 5)
#define low CRGB(0, 0, 0)
```

CRGB Object Array

The LED strip to be controlled was declared as leds, while the musical score to be followed was declared as score. Both were defined as two-dimensional arrays of CRGB objects.

Within the main function, the elements of leds were sequentially mapped to the corresponding elements in score. Through this mapping process, the LED strip was able to illuminate according to the musical notes and rhythm defined in the score, achieving synchronized visual playback of the music sequence.

```
CRGB leds[keyNUM][NUM_LEDS] = { low };
CRGB score[keyNUM][scoreNumber] = { { low, low, low, low, low, low, low, low, highd
                                    { low, low, low, low, low, low, low, highr, low
                                    { highm, highm, low, low, low, low, highm, low,
                                    { low, low, highf, low, low, highf, low, low, l
                                    { low, low, low, highs, highs, low, low, low, l
                                    { low, low, low, low, low, low, low, low, low,
                                    { low, low, low, low, low, low, low, low, low,
```

Main Loop Functions

The start_func and end functions are used to notify the player when the music begins and ends. In the current design, all LEDs flash twice to indicate the start and end of the game.

In future improvements, more dynamic visual effects can be implemented. For example, at the beginning, the seven rows of LED strips could illuminate sequentially to create a rainbow effect. At the end, the LED strip could directly display the final score.

The led function serves as the core function that controls the LED strip according to the musical score and creates the visual effect of the light signals falling from the top to the bottom row.

The program logic repeatedly executes the following operations using a loop:

First, the first row of the leds array is assigned the corresponding value from the score array. Then, each subsequent row of the leds array is assigned the value of the previous row.

By continuously repeating this process until the score sequence ends, the LED strip illuminates according to the musical notes and rhythm while creating a downward “falling” effect.

The advantage of this programming structure is its scalability. To add a new song, it is only necessary to define a new CRGB array representing that song’s score. The same function can then be reused to achieve the corresponding LED visualization without modifying the core control logic.

```
void loop() {
  start = digitalRead(A0);
  while (start == 1) {
    // pace = analogRead(pace_PIN);
    start_func();
    led();
    end();
    start = 0;
  }
}
```

4. Scoring System

Scoring System

Design Concept:
When the LED signal reaches the bottom row, the user must press the corresponding piano key. During gameplay, the system calculates the number of times the user presses the key at the correct timing and displays the accumulated score.

Implementation Method:

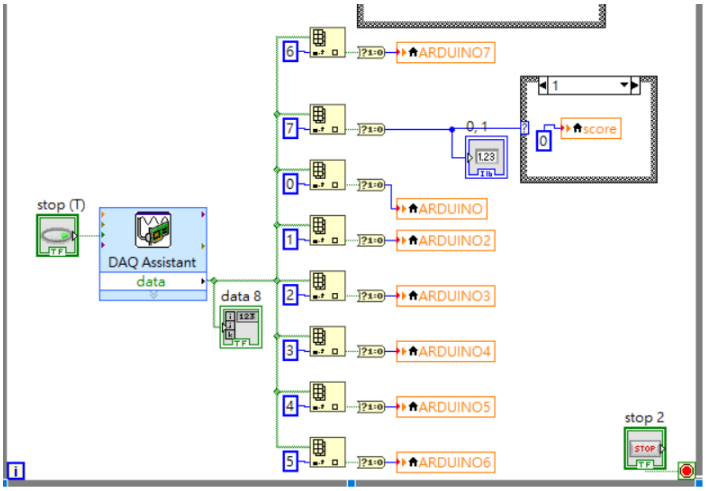
When the last row of the leds array is illuminated (a value is assigned in the program), a high-level voltage signal is transmitted to the NI ELVIS system. The signal is then acquired using the DAQ Assistant in LabVIEW.

This signal is evaluated together with the voltage signal from the corresponding touch sensor of the piano key. If both signals are detected as high at the same time, the score is incremented by one. Otherwise, no point is awarded.

At the same time, the real-time score is displayed using the seven-segment display.

Program Integration:

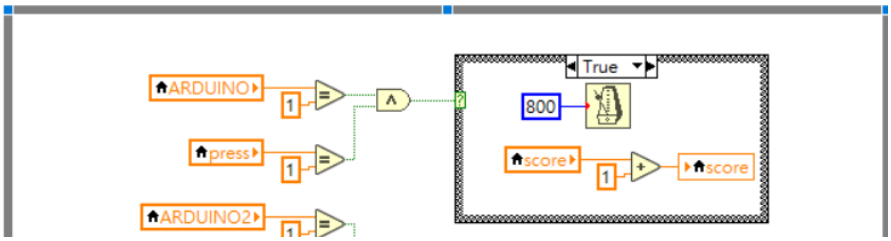
Arduino–ELVIS Connection:
The DAQ Assistant in LabVIEW is used to receive the voltage signal transmitted from the Arduino and store it into a variable for subsequent logical processing and scoring evaluation.



Scoring Logic:

The system determines whether the voltage signal from the Arduino and the signal from the touch sensor switch are both at a high logic level simultaneously. If both signals are high at the same time, the score is incremented by one.

After a valid score is recorded, the system waits for 800 milliseconds. This delay corresponds to the rhythm interval of the LED signal and prevents repeated scoring within the same beat cycle.

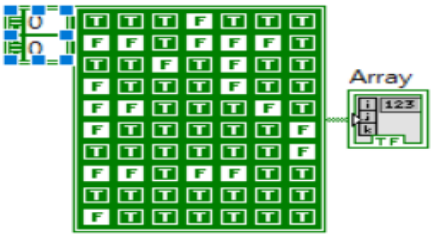


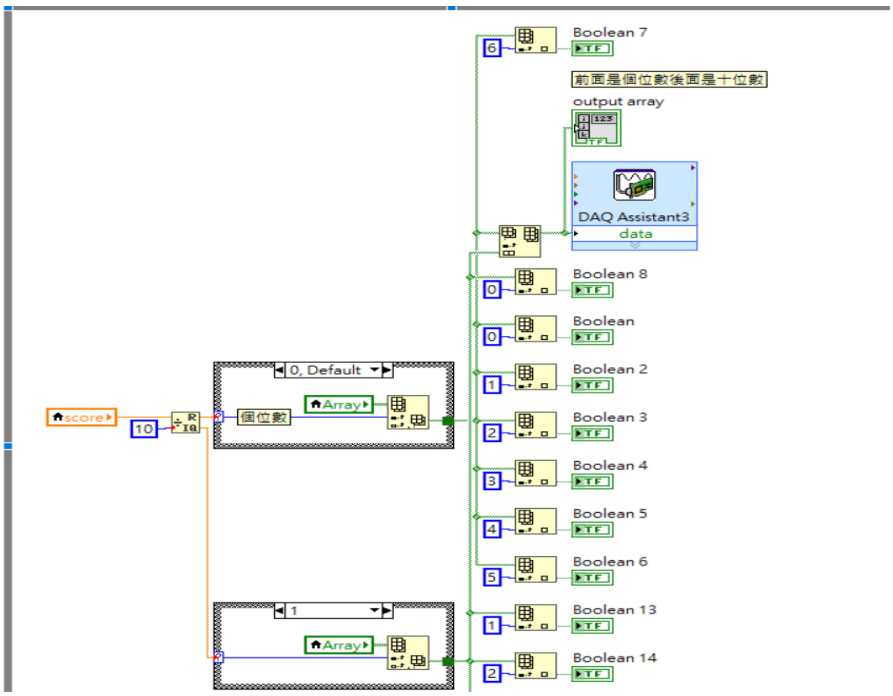
Seven-Segment Display for Score Output

The seven-segment display is used to present the real-time score during gameplay.

In the program, the illumination patterns corresponding to digits 0 through 9 were first defined by specifying the on/off states of each segment.

Based on the current score value, the DAQ Assistant in LabVIEW is used to transmit the appropriate voltage signals to each segment pin, thereby displaying the correct numerical digit on the seven-segment display.





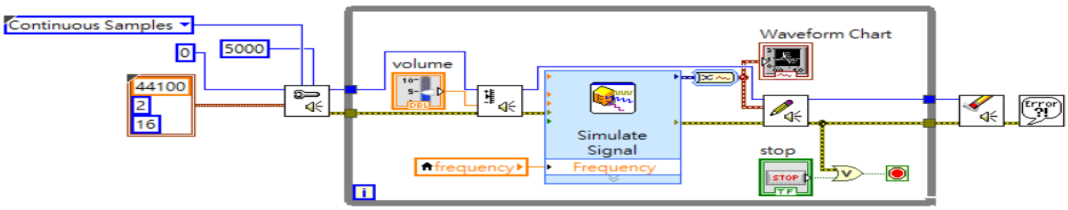
Sound Output for Key Press

Design Concept:
The system detects which piano key is pressed and generates the corresponding sound. This allows the player to hear the melody being played and verify whether the performance matches the intended notes during gameplay.

Implementation Method:
A capacitive touch sensor is installed beneath each piano key. When the bottom of a key makes contact with the sensor, a high-level voltage signal is sent to the NI ELVIS system. The signal is acquired using the DAQ Assistant module in LabVIEW.

By identifying which input channel receives the high-level signal, the system determines which piano key is currently pressed. A corresponding audio frequency is then assigned and transmitted to the computer, allowing the computer speaker to output the correct pitch for that key.

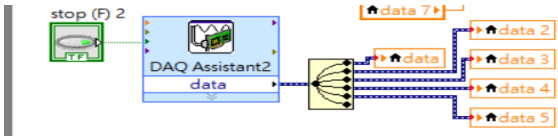
Programming Implementation:
Fixed audio parameters were configured in the program (sample rate = 44,100 Hz, number of channels = 2, bits per sample = 16). A variable frequency was then defined to adjust the output tone frequency, enabling the generation of notes from Do to Si.



Signal Acquisition and Channel Separation

The DAQ Assistant in LabVIEW was used to acquire the key press signals.

To enable a single DAQ Assistant module to receive signals from multiple input channels simultaneously, the Split Signal function was implemented. This function separates the acquired multi-channel signal into individual channels, allowing independent processing and identification of each piano key input.



Based on the received key press signal, the system determines which note is played and assigns the corresponding frequency.

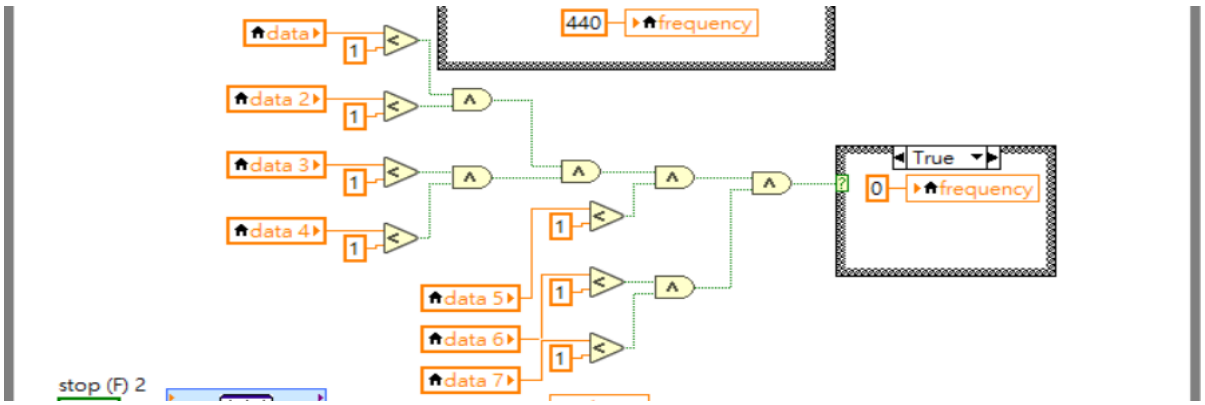
5. Challenges Encountered and Solutions

1. Sustain Issue:

After a key is pressed, the audio signal is generated but does not stop, resulting in a sustained sound similar to holding a piano pedal. The sound cannot terminate properly.

Solution:

Multiple AND conditions were implemented to detect when no key press signal is received. When no input signal is detected, the variable frequency is forced to zero to prevent it from retaining the value of the previous key press.

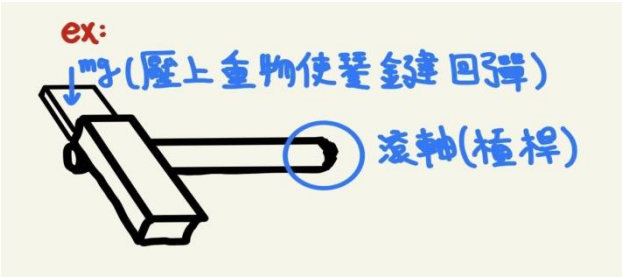


2. Key Structure Issue:

The original key design, as shown in the figure, utilized a lever mechanism to achieve a return effect. However, this design caused instability issues. During gameplay, the keys could shift vertically or horizontally due to repeated user interaction.

Solution:

The design was modified by cutting the corrugated board and integrating an elastic rubber band mechanism. This improved structural stability while maintaining the key return function. The detailed structure can be found in the Key Return Mechanism section of the Structural Design chapter.



3. Repeated Scoring Issue:

In some cases, a single key press was recorded as two or more points.

Solution:

A wait function was introduced to create a short delay in the NI ELVIS system. This ensures that scoring cannot occur again before the next LED signal reaches the bottom row. The implementation details can be found in the Scoring section of the program.

---

4. Excessive and Disorganized Wiring Issue:

Due to the large number of sensors and the integration of Arduino, the wiring became complex and prone to accidental contact or loose connections.

Solution:

A breadboard was used to centralize and organize the circuit connections between the Arduino and NI ELVIS. Wires with similar functions were grouped and secured with adhesive tape. For unstable DuPont wire connections, soldering was applied to ensure reliable electrical conductivity.

---

5. LED Strip Height Issue:

In the original design, the LED strip was positioned at a higher location, as shown in the figure. However, testing revealed that this configuration was inconvenient for users. Additionally, identical colors among the LEDs caused visual confusion and reduced gameplay clarity.

Solution:

The LED strip was repositioned directly in front of the user to improve visibility and usability. A dark-colored corrugated board was used as the base, with holes cut to embed the LED strip for a cleaner and more focused visual presentation.

Additionally, different colors were assigned to each row to reduce visual confusion and enhance clarity during gameplay. The final configuration can be referenced in the completed assembly shown in Figure 9 in the Touch Sensor and Start Button section.

6. Conclusion

The project successfully implemented an LED strip that visually follows a manually programmed musical score (Ode to Joy). LabVIEW was able to receive the signal from the final LED transmitted by the Arduino. When a key was pressed, the system correctly determined whether the user scored and generated the corresponding piano sound.

The seven-segment display recorded the user's score with minimal error. Through this integration of hardware and software, a gameplay concept similar to mobile rhythm games was successfully realized using a simulated physical piano interface.

Future Work

Adjustable Tempo Control

A variable resistor can be incorporated to adjust the falling speed of the LED strip, thereby controlling the rhythm tempo. This feature was initially tested; however, to resolve the repeated scoring issue, a fixed LED falling speed and wait function were adopted.

In future improvements, conditional logic can be implemented in LabVIEW to detect the resistance value and dynamically adjust the wait time. This would allow difficulty adjustment based on tempo control.

Melody Recording Function

By reversing the signal flow and acquiring touch sensor inputs through NI ELVIS, the sequence and duration of key presses can be recorded in LabVIEW. This would enable a memory function where users can play and record a melody.

The recorded melody could then be transmitted back to the Arduino, allowing the LED strip to replay the custom score for another user to perform.

LED-Based Score Display

The final score could be displayed directly using the LED strip as an alternative visual output, replacing or supplementing the seven-segment display.

Expansion to Full Octave with Black Keys

Black keys can be added with an improved mechanical structure to prevent interference between black and white keys. By integrating additional touch sensors, the system could be expanded to support a full octave keyboard.